🎓 Placement Management System

Complete Technical Documentation

Version 1.0.0

🎓 Placement Management System - Complete Documentation

📋 Table of Contents

1. Project Overview
2. Technology Stack
3. Project Structure
4. Environment Setup
5. Database Schema
6. API Documentation
7. Frontend Components
8. Authentication & Authorization
9. File Upload System
10. Deployment Guide
11. Development Workflow
12. Troubleshooting

🎯 Project Overview

The **Placement Management System** is a full-stack web application designed to manage student profiles, academic records, and placement-related information for educational institutions. The system provides separate interfaces for students and administrators, with features for profile management, academic tracking, resume management, and administrative oversight.

▶ Key Features

- **Student Portal**: Profile management, academic records, resume building
- **Admin Portal**: Student management, department oversight, analytics
- **Image Management**: Cloudinary integration for profile photos
- **Responsive Design**: Modern UI with Tailwind CSS
- **Secure Authentication**: JWT-based authentication system

🛠 Technology Stack

▶ Backend

- **Node.js** - Runtime environment
- **Express.js** - Web framework
- **MongoDB** - Database
- **Mongoose** - ODM for MongoDB
- **JWT** - Authentication tokens
- **bcryptjs** - Password hashing
- **Cloudinary** - Image storage and management
- **Multer** - File upload handling

▶ Frontend

- **React 18** - UI library
- **TypeScript** - Type safety
- **Vite** - Build tool and dev server
- **React Router** - Client-side routing
- **Tailwind CSS** - Styling framework
- **Axios** - HTTP client
- **Framer Motion** - Animations

▶ Development Tools

- **Nodemon** - Server auto-restart
- **PostCSS** - CSS processing
- **Autoprefixer** - CSS vendor prefixes

📁 Project Structure

```
placement-app/
├── client/                    # React frontend
│   ├── src/
│   │   ├── components/        # Reusable components
│   │   │   ├── ErrorBoundary.tsx
│   │   │   ├── Footer.tsx
│   │   │   └── ProtectedRoute.tsx
│   │   ├── pages/             # Page components
│   │   │   ├── admin/         # Admin pages
│   │   │   │   ├── AdminLogin.tsx
│   │   │   │   ├── Dashboard.tsx
│   │   │   │   ├── Departments.tsx
│   │   │   │   ├── StudentDetails.tsx
│   │   │   │   └── StudentList.tsx
│   │   │   ├── EditProfile.tsx
│   │   │   ├── Login.tsx
│   │   │   ├── Profile.tsx
│   │   │   ├── Register.tsx
│   │   │   ├── Resume.tsx
│   │   │   └── Semester.tsx
│   │   ├── styles/            # Custom CSS
│   │   │   ├── legacy-login.css
│   │   │   └── register.css
│   │   ├── lib/               # Utilities
│   │   │   └── api.ts
│   │   ├── index.css
│   │   └── main.tsx
│   ├── dist/                  # Build output
│   ├── package.json
│   ├── tailwind.config.js
│   ├── tsconfig.json
│   └── vite.config.ts
├── server/                    # Node.js backend
│   ├── lib/                   # Utilities
│   │   ├── cloudinary.js
│   │   ├── db.js
│   │   └── seed.js
│   ├── middleware/            # Express middleware
│   │   └── auth.js
│   ├── models/                # Mongoose models
│   │   ├── Admin.js
│   │   └── Student.js
│   ├── routes/                # API routes
│   │   ├── admin.js
│   │   ├── auth.js
│   │   ├── students.js
```

```
│   │   └── upload.js
│   └── index.js              # Server entry point
├── .env                      # Environment variables
├── package.json              # Root package.json
├── nodemon.json
├── Procfile                  # Deployment config
└── README.md
```

⚙️ Environment Setup

▶ Prerequisites

- Node.js (>= 18)
- MongoDB Atlas account
- Cloudinary account

▶ 1. Clone and Install Dependencies

```
git clone <repository-url>
cd placement-app
npm install
```

▶ 2. Environment Variables

Create a `.env` file in the root directory:

```
# Database
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/placement-app

# JWT Secret
JWT_SECRET=your_jwt_secret_here

# Client Origin
CLIENT_ORIGIN=http://localhost:5173

# Cloudinary Configuration
CLOUDINARY_URL=cloudinary://api_key:api_secret@cloud_name

# Server Port
PORT=10000

# Admin Credentials (Optional)
ADMIN_USERNAME=aurccadmin
ADMIN_PASSWORD=placementaurcc
```

▶ 3. Start Development Servers

```
# Start backend server
npm run dev

# Start frontend (in new terminal)
npm run client:dev
```

🗄 Database Schema

▶ Student Model

```
{
  // Basic Information
  name: String (required)
  registerNumber: String (required, unique)
  email: String (required, unique)
  passwordHash: String (required)
  department: String (required)
  year: String (required)

  // Profile Information
  profilePhoto: String
  profilePhotoPublicId: String
  dob: Date
  age: Number
  gender: String
  collegeName: String
  collegeAddress: String
  address: String
  phone: String
  currentSemester: Number (1-8)

  // Academic Information
  academic: {
    cgpa: Number
    sgpa: [Number]
    semesters: [{
      semesterNumber: Number
      subjects: [{
        subjectName: String
        subjectCode: String
        credits: Number
        grade: String
        points: Number
      }]
      sgpa: Number
      totalCredits: Number
      earnedCredits: Number
    }]
    historyOfArrears: Number
    currentArrears: Number
    hscPercentage: Number
    sslcPercentage: Number
    dateOfEntry: Date
```

```
    status: String (Active/Inactive)
    currentSemester: Number (1-8)
  }

  // Placement Information
  placement: {
    achievements: String
    internships: String
    workExperience: String
    projects: String
    certifications: String
    technicalSkills: [String]
    logicalSkills: [String]
    willingToPlace: Boolean
    placementPreference: String
  }

  // Links
  links: {
    resume: String
    portfolio: String
    linkedin: String
    github: String
  }

  // Timestamps
  createdAt: Date
  updatedAt: Date
}
```

▶ Admin Model

```
{
  username: String (required, unique)
  passwordHash: String (required)
  role: String (default: 'admin')
  createdAt: Date
  updatedAt: Date
}
```

API Documentation

Base URL

```
http://localhost:10000/api
```

Authentication Endpoints

POST `/AUTH/STUDENT/REGISTER`

Register a new student

```
{
  "name": "John Doe",
  "registerNumber": "21CS001",
  "email": "john@example.com",
  "password": "password123",
  "department": "CSE",
  "year": "Final"
}
```

POST `/AUTH/STUDENT/LOGIN`

Student login

```
{
  "registerNumber": "21CS001",
  "password": "password123"
}
```

POST `/AUTH/ADMIN/LOGIN`

Admin login

```
{
  "username": "aurccadmin",
  "password": "placementaurcc"
}
```

POST `/AUTH/ADMIN/SEED`

Create default admin (idempotent)

▶ Student Endpoints

GET `/STUDENTS/ME`

Get current student profile

- **Headers**: `Authorization: Bearer <token>`
- **Role**: student

PUT `/STUDENTS/ME`

Update current student profile

- **Headers**: `Authorization: Bearer <token>`
- **Role**: student
- **Body**: Student object (partial update supported)

GET `/STUDENTS`

List students (admin only)

- **Headers**: `Authorization: Bearer <token>`
- **Role**: admin
- **Query**: `?year=Final&department=CSE`

GET `/STUDENTS/:ID`

Get student by ID

- **Headers**: `Authorization: Bearer <token>`
- **Role**: admin

PUT `/STUDENTS/:ID`

Update student by ID (admin only)

- **Headers**: `Authorization: Bearer <token>`

- **Role**: admin

▶ Admin Endpoints

GET `/ADMIN/YEARS`

Get available years

- **Headers**: `Authorization: Bearer <token>`
- **Role**: admin
- **Response**: `["First", "Second", "Third", "Final"]`

GET `/ADMIN/DEPARTMENTS`

Get available departments

- **Headers**: `Authorization: Bearer <token>`
- **Role**: admin
- **Response**: `["CSE", "AI&DS", "Mech", "ECE", "EEE", "VLSI"]`

GET `/ADMIN/STUDENTS`

Get students with filters

- **Headers**: `Authorization: Bearer <token>`
- **Role**: admin
- **Query**: `?year=Final&department=CSE`

DELETE `/ADMIN/STUDENTS/:ID`

Delete student

- **Headers**: `Authorization: Bearer <token>`
- **Role**: admin

DELETE `/ADMIN/STUDENTS/:ID/PROFILE-PHOTO`

Remove student's profile photo

- **Headers**: `Authorization: Bearer <token>`
- **Role**: admin

▶ Upload Endpoints

POST `/UPLOAD/PROFILE-PHOTO`

Upload profile photo

- **Headers**: `Authorization: Bearer <token>`
- **Role**: student
- **Content-Type**: `multipart/form-data`
- **Body**: `profilePhoto` (file)

DELETE `/UPLOAD/PROFILE-PHOTO`

Remove profile photo

- **Headers**: `Authorization: Bearer <token>`
- **Role**: student

🎨 Frontend Components

▶ Student Pages

LOGIN (`/LOGIN`)

- Student authentication form
- Register number and password
- Redirects to profile on success

REGISTER (`/REGISTER`)

- Student registration form
- All required fields validation
- Redirects to login on success

PROFILE (`/PROFILE`)

- View student profile information
- Navigation to edit, semester, resume pages
- Display profile photo

EDIT PROFILE (`/EDIT`)

- Comprehensive profile editing form
- Image upload with Cloudinary integration
- Academic and placement information

- Form validation and error handling

SEMESTER (`/PROFILE/SEMESTER`)

- Academic semester management
- Subject and grade entry
- SGPA calculation

RESUME (`/PROFILE/RESUME`)

- Resume building interface
- Links and portfolio management
- Skills and experience entry

▶ Admin Pages

ADMIN LOGIN (`/ADMIN/LOGIN`)

- Admin authentication
- Username and password

DASHBOARD (`/ADMIN/DASHBOARD`)

- Year selection interface
- Navigation to departments and students
- Logout functionality

DEPARTMENTS (`/ADMIN/DEPARTMENTS`)

- Department-wise student management
- Filter by year and department
- Student statistics

STUDENT LIST (`/ADMIN/STUDENTS`)

- List all students with filters
- Search and pagination
- Quick actions

STUDENT DETAILS (`/ADMIN/STUDENTS/:ID`)

- Detailed student view
- Edit student information

- Delete student functionality

▶ Components

PROTECTEDROUTE

- Route protection based on user role
- Redirects unauthorized users
- Token validation

ERRORBOUNDARY

- React error boundary
- Graceful error handling
- User-friendly error messages

FOOTER

- Consistent footer across pages
- Navigation links

🔐 Authentication & Authorization

▶ JWT Token Structure

```
{
  "sub": "user_id",
  "role": "student" | "admin",
  "iat": 1234567890,
  "exp": 1234567890
}
```

▶ Token Storage

- **Frontend**: localStorage
- **Backend**: Bearer token in Authorization header

▶ Role-Based Access

- **Student**: Can access own profile, edit information
- **Admin**: Can access all student data, manage users

▶ Password Security

- **Hashing**: bcryptjs with salt rounds 10
- **Validation**: Server-side validation required

📁 File Upload System

▶ Cloudinary Integration

- **Storage**: Cloudinary cloud storage
- **Processing**: Automatic image optimization
- **Transformations**: 400×400 resize, face detection
- **Folder**: `placement-app/profile-photos`

▶ Upload Flow

1. User selects image file
2. Frontend validates file type and size
3. File sent to `/upload/profile-photo`
4. Multer processes file with Cloudinary storage
5. Previous image deleted from Cloudinary
6. New image URL stored in database
7. Frontend updates with new image

▶ File Validation

- **Types**: JPG, PNG, GIF, WebP
- **Size**: Maximum 5MB
- **Processing**: Automatic optimization

🚀 Deployment Guide

Environment Setup

1. **MongoDB Atlas**: Create cluster and get connection string
2. **Cloudinary**: Create account and get credentials
3. **Environment Variables**: Set all required variables

Build Process

```
# Install dependencies
npm install

# Build frontend
npm run client:build

# Start production server
npm start
```

Render.com Deployment

1. Connect GitHub repository
2. Set environment variables
3. Deploy automatically on push

Environment Variables for Production

```
MONGODB_URI=mongodb+srv://...
JWT_SECRET=strong_secret_key
CLIENT_ORIGIN=https://your-domain.com
CLOUDINARY_URL=cloudinary://...
PORT=10000
```

🔄 Development Workflow

▶ Starting Development

```
# Terminal 1: Backend
npm run dev

# Terminal 2: Frontend
npm run client:dev
```

▶ Code Structure

- **Backend**: Express.js with modular routes
- **Frontend**: React with TypeScript
- **Styling**: Tailwind CSS with custom components
- **State Management**: React hooks and context

▶ Git Workflow

1. Create feature branch
2. Make changes
3. Test locally
4. Commit and push
5. Create pull request

🐛 Troubleshooting

▶ Common Issues

1. PORT ALREADY IN USE

```
# Find process using port 10000
netstat -ano | findstr :10000

# Kill process
taskkill /PID <PID> /F
```

2. MONGODB CONNECTION ISSUES

- Check connection string format
- Verify network access in MongoDB Atlas
- Check database name in connection string

3. CLOUDINARY UPLOAD FAILURES

- Verify CLOUDINARY_URL format
- Check API credentials
- Ensure proper file format and size

4. JWT TOKEN ISSUES

- Verify JWT_SECRET is set
- Check token expiration
- Ensure proper token format

5. CORS ISSUES

- Check CLIENT_ORIGIN setting
- Verify frontend URL matches

▶ Debug Mode

```
# Enable debug logging
DEBUG=* npm run dev
```

▶ Logs

- **Server**: Console output with morgan logging
- **Database**: Mongoose connection logs
- **Upload**: Cloudinary operation logs

📊 Project Statistics

- **Total Files**: 25+ source files
- **API Endpoints**: 15+ endpoints
- **React Components**: 10+ components
- **Database Models**: 2 models
- **Authentication**: JWT-based
- **File Storage**: Cloudinary integration
- **Responsive Design**: Mobile-first approach

🔮 Future Enhancements

▶ Planned Features

- [ ] Email notifications
- [ ] Advanced search and filtering
- [ ] Export functionality (PDF, Excel)
- [ ] Real-time notifications
- [ ] Mobile app
- [ ] Analytics dashboard
- [ ] Bulk operations
- [ ] Advanced reporting

▶ Technical Improvements

- [ ] Unit testing
- [ ] Integration testing
- [ ] Performance optimization
- [ ] Caching layer
- [ ] API rate limiting
- [ ] Enhanced security

Support

For technical support or questions:

1. Check this documentation
2. Review error logs
3. Check environment variables
4. Verify database connectivity
5. Test API endpoints

This documentation covers the complete Placement Management System from setup to deployment. Keep it updated as the project evolves.

Placement Management System - Professional Documentation
Generated on September 12, 2025